

Exemple, LeSas (v1)

Fabrice.Kordon@lip6.fr

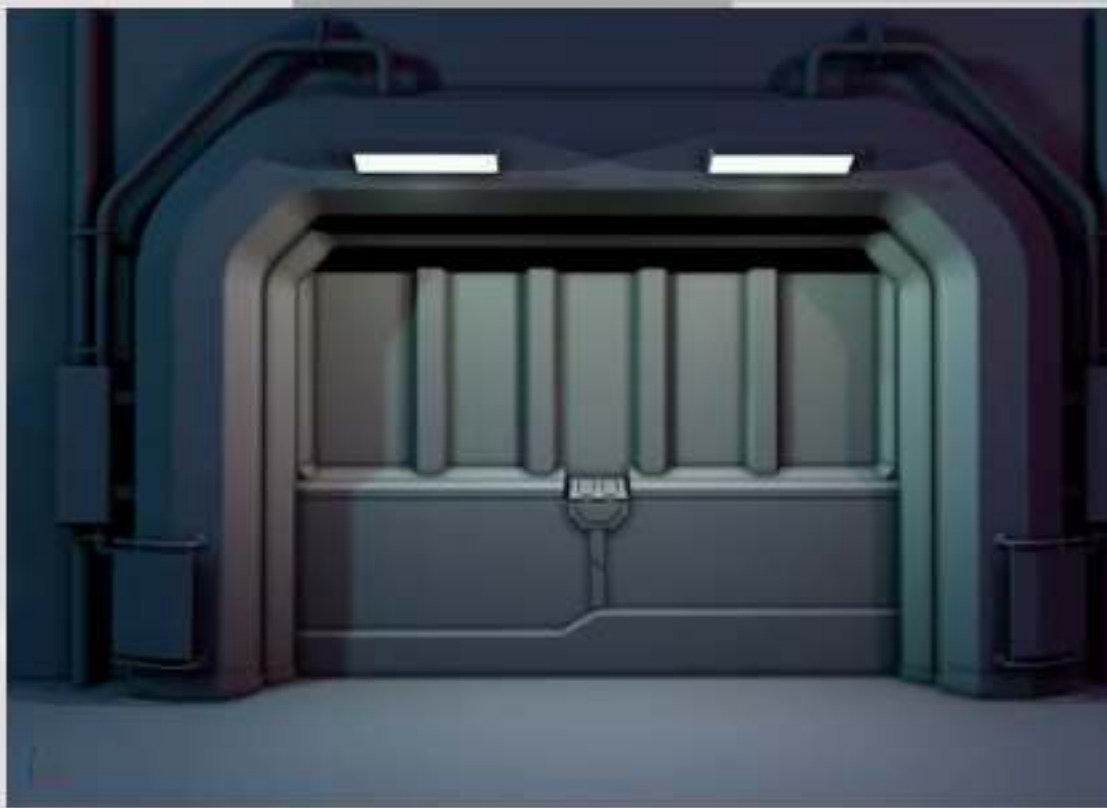


Fonctionnement d'un Sas

2

Capacité fixée

-  Laisser passer les «Passagers» par groupe de N



etc.



Points délicats

-  Blocage sur une entrée (lorsque le sas est plein)
-  Blocage sur une sortie (lorsque le sas se remplit)
-  Ne pas laisser le sas se reremplir tant qu'il n'est pas vidé
 - Une seule porte ouverte à la fois

La classe Passenger (v1)

3

```
class Passenger1 implements Runnable {
    private Sas1 s;
    private int monId;
    private static final Object mutex = new Object();
    private static int cpt = 1;

    Passenger1 (Sas1 s) {
        synchronized (mutex) {
            monId = cpt++;
        }
        this.s = s;
    }

    private void trace (String m) {
        System.out.println ("Passager " + monId + " : " + m);
        Thread.yield(); // rendre la commutation possible
    }

    public void run() {
        try {
            this.trace ("prêt à entrer");
            s.entrerDansSas(monId);
            this.trace ("j'y suis");
            s.sortirDuSas(monId);
            this.trace ("sorti");
        }
        catch (InterruptedException e) {
            this.trace ("passager " + monId + " interrompu " + e);
        }
    }
}
```


La classe Sas (v1)

données & constructeur

4

```
class Sas1 {
    private final int capacite; // capacité du sas
    private int contenu; // nombre de places libres
    private boolean remplir; // true = phase de remplissage

    private final Lock l = new ReentrantLock(); // moniteur
    private final Condition entrer = l.newCondition(); // condition pour l'entrée
    private final Condition sortir = l.newCondition(); // condition pour la sortie

    Sas1 (int taille) {
        capacite = taille;
        contenu = taille;
        remplir = true; // le sas doit se remplir
    }
}
```


La classe Sas (v1)

méthodes d'accès

5

```
public void entrerDansSas (int qui) throws InterruptedException {  
    l.lock();  
    try {  
        while (!remplir) {  
            entrer.await();  
        }  
        contenu--;  
        if (contenu == 0) {  
            System.out.println (" " + qui + " dedans + ferme l'entrée");  
            remplir = false;  
            sortir.signalAll(); // sas plein, réveil coté sortie  
        } else {  
            System.out.println (" " + qui + " dedans");  
        }  
    }  
    finally {  
        l.unlock(); // toujours exécuté  
    }  
}
```


La classe Sas (v1) méthodes d'accès

5

Simple non?

Transcription plus fidèle d'un
moniteur de Hoare

```
public void entrerDansSas (int qui) throws InterruptedException {  
    l.lock();  
    try {  
        while (!remplir) {  
            entrer.await();  
        }  
        contenu--;  
        if (contenu == 0) {  
            System.out.println (" " + qui + " dedans + ferme l'entrée");  
            remplir = false;  
            sortir.signalAll(); // sas plein, réveil coté sortie  
        } else {  
            System.out.println (" " + qui + " dedans");  
        }  
    }  
    finally {  
        l.unlock(); // toujours exécuté  
    }  
}
```

```
public void sortirDuSas (int qui) throws InterruptedException {  
    l.lock();  
    try {  
        while (remplir) {  
            System.out.println (" " + qui + " devant la sortie");  
            sortir.await();  
        }  
        contenu++;  
        if (contenu == capacite) {  
            System.out.println (" " + qui + " ferme la sortie");  
            remplir = true;  
            entrer.signalAll(); // sas vide, réveil coté entrée  
        } else {  
            System.out.println (" " + qui + " sort");  
        }  
    }  
    finally {  
        l.unlock();  
    }  
}
```


La classe principale

6

```
public class LeSas1 {  
    public static void main(String []args) {  
        final int cstMaxPassagers;  
        final int cstCapaciteSas;  
  
        try {  
            cstCapaciteSas = Integer.parseInt(args[0]);  
            cstMaxPassagers = Integer.parseInt(args[1]);  
        }  
        catch (ArrayIndexOutOfBoundsException e) {  
            System.err.println ("Arguments requis, capacité du sas et nombre de passagers.");  
            return;  
        }  
  
        Sas1 leSas = new Sas1(cstCapaciteSas);  
  
        for (int i = 0; i < cstMaxPassagers ; i++) {  
            new Thread (new Passenger1 (leSas)).start();  
        }  
    }  
}
```


Quelques exécutions

7

```
java LeSas1 2 3
Passager 1 : prêt à entrer
Passager 3 : prêt à entrer
Passager 2 : prêt à entrer
1 dedans
Passager 1 : j'y suis
2 dedans + ferme l'entrée
Passager 2 : j'y suis
1 sort
Passager 1 : sorti
2 ferme la sortie
Passager 2 : sorti
3 dedans
Passager 3 : j'y suis
3 devant la sortie
```

```
$ java LeSas1 3 3
Passager 1 : prêt à entrer
Passager 3 : prêt à entrer
Passager 2 : prêt à entrer
1 dedans
Passager 1 : j'y suis
3 dedans
Passager 3 : j'y suis
2 dedans + ferme l'entrée
Passager 2 : j'y suis
1 sort
Passager 1 : sorti
3 sort
Passager 3 : sorti
2 ferme la sortie
Passager 2 : sorti
```

```
$ java LeSas1 2 4
Passager 1 : prêt à entrer
Passager 4 : prêt à entrer
Passager 2 : prêt à entrer
Passager 3 : prêt à entrer
1 dedans
Passager 1 : j'y suis
4 dedans + ferme l'entrée
Passager 4 : j'y suis
1 sort
Passager 1 : sorti
4 ferme la sortie
Passager 4 : sorti
2 dedans
Passager 2 : j'y suis
3 dedans + ferme l'entrée
Passager 3 : j'y suis
3 sort
Passager 3 : sorti
2 ferme la sortie
Passager 2 : sorti
```

Question

Que se passe-t-il sur cette
3^{ème} exécution?

Analysons l'exécution fautive

8



Ceci explique le blocage!

```
$ java LeSas1 2 3
Passager 1 : prêt à entrer
Passager 3 : prêt à entrer
Passager 2 : prêt à entrer
  1 dedans
Passager 1 : j'y suis
  2 dedans + ferme l'entrée
Passager 2 : j'y suis
  1 sort
Passager 1 : sorti
  2 ferme la sortie
Passager 2 : sorti
  3 dedans
Passager 3 : j'y suis
  3 devant la sortie
```

^C

Variante avec «effet domino»

9

```
public void entrerDansSas (int qui) throws InterruptedException {
    l.lock();
    try {
        while (!remplir) {
            entrer.await();
        }
        contenu--;
        if (contenu == 0) {
            System.out.println (" " + qui + " dedans + ferme l'entrée");
            remplir = false;
            sortir.signal(); // sas plein, réveil coté sortie
        } else {
            System.out.println (" " + qui + " dedans");
            entrer.signal(); // Effet domino
        }
    }
    finally {
        l.unlock(); // toujours exécuté
    }
}
```


Variante avec «effet domino»

```
public void sortirDuSas (int qui) throws InterruptedException {
    l.lock();
    try {
        while (remplir) {
            System.out.println (" " + qui + " devant la sortie");
            sortir.await();
        }
        contenu++;
        if (contenu == capacite) {
            System.out.println (" " + qui + " ferme la sortie");
            remplir = true;
            entrer.signal(); // sas vide, réveil coté entrée
        } else {
            System.out.println (" " + qui + " sort");
            sortir.signal(); // Effet domino
        }
    }
    finally {
        l.unlock();
    }
}
```


Variante avec «effet domino»

9



Avant

le premier qui sort
réveille tous les autres

```
public void sortirDuSas (int qui) throws InterruptedException {
    l.lock();
    try {
        while (remplir) {
            System.out.println (" " + qui + " devant la sortie");
            sortir.await();
        }
        contenu++;
        if (contenu == capacite) {
            System.out.println (" " + qui + " ferme la sortie");
            remplir = true;
            entrer.signal(); // sas vide, réveil coté entrée
        } else {
            System.out.println (" " + qui + " sort");
            sortir.signal(); // Effet domino
        }
    }
    finally {
        l.unlock();
    }
}
```


Variante avec «effet domino»



Avant

le premier qui sort
réveille tous les autres



Maintenant

le premier qui sort
réveille le suivant
qui réveille le suivant...

```
public void sortirDuSas (int qui) throws InterruptedException {  
    l.lock();  
    try {  
        while (remplir) {  
            System.out.println (" " + qui + " devant la sortie");  
            sortir.await();  
        }  
        contenu++;  
        if (contenu == capacite) {  
            System.out.println (" " + qui + " ferme la sortie");  
            remplir = true;  
            entrer.signal(); // sas vide, réveil coté entrée  
        } else {  
            System.out.println (" " + qui + " sort");  
            sortir.signal(); // Effet domino  
        }  
    }  
    finally {  
        l.unlock();  
    }  
}
```


En guise de conclusion

10

ReentrantLock

-  Mise en œuvre plus fidèle des moniteurs de Hoare
 - ▶ Plus riche à manipuler que le moniteur intégré aux objets
-  Encore des points délicats
 - ▶ Priorités de reprise

N'hésitez pas à jouer sur des «effets domino»

-  C'est un classique de la discipline
-  Mais Java recommande de ne pas y jouer

